

Software Design Document

Secure Deep Scan



Submitted by

Hannan Fiaz
S23BINFT1M01112

Submitted to

Dr. Musarat Kareem

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Summary

The Secure Deep Scan System is a web based platform designed to scan websites for common web application vulnerabilities and security attacks using the OWASP ZAP API. The system helps users identify vulnerabilities such as SQL Injection (SQLi), Cross-Site Scripting (XSS), broken authentication, security misconfigurations, and other OWASP Top 10 vulnerabilities through automated website scanning and report generation. The system allows users to submit website URLs for security scanning, perform automated vulnerability analysis, view detailed scan reports, and monitor scan history through a secure dashboard. The application follows MVC architecture using Django and MySQL to ensure scalability, maintainability, and secure data handling. The project purpose to provide an efficient, intelligent, and user-friendly cybersecurity solution for secure against digital growing attacks on Internet.

Table of Contents

1. Application Architecture	4
1.1 MVC Architecture	4
1.2 Proposed Models	4
1.3 Proposed Views	5
1.4 Proposed Controllers	5
1.4.1 Website Scan Controller	5
1.4.2 Notification Controller	6
1.5 APIs and External Libraries	6
1.6 UML Diagrams	6
1.6.1 Level Design (HLD) Architecture Diagram	7
1.7 Coding Conventions	8
2. Data Model Schema	8
2.1 Entities	8
2.1.1 Website Scan Entity	9
2.1.2 Scan Report Entity	9
2.1.3 Threat Detection Entity	9
2.1.4 System Log Entity	10
2.1.5 Entity Relationships	10
2.2 Database Constraints	10
2.3 Data Integrity	11
2.4 Future Database Enhancements	11
3. User Interface	12
3.1 User Interface Design Philosophy	12
3.2 Color Scheme	12
3.3 Typography	13
3.4 Responsiveness and Screen Constraints	13
3.5 User Interface Screens	14
3.5.1 Home Page	14
3.5.2 Navigation Page	15
3.5.3 Report View Page	16

3.9 Future UI Enhancements.....	17
4. References	17

1. Application Architecture

The application follows a modular, decoupled **MVC** designed to separate the user interface from the background computing logic. The frontend provides an intuitive web interface for configuring audits and visualizing results, communicating via an internal REST API with a **Django web backend**. To ensure high availability and responsiveness, time-consuming security audits are handed off asynchronously to a multi-threaded **vulnerability scanning engine** managed by a background task queue. Finally, a persistent **MySQL database** enforces strict relational integrity, safely storing account configurations, discovered attack surfaces, and historical vulnerability metrics without blocking the active web server.

1.1 MVC Architecture

Component	Purpose
Model	Handles database operations
View	Displays graphical user interface
Controller	Processes requests and business logic

1.2 Proposed Models

- User
- WebsiteScan
- ScanReport
- ThreatDetection

1.3 Proposed Views

Views

- Dashboard View
- Website Scan View
- Scan Report View

1.4 Proposed Controllers

Proposed Controller are controllers which will be used in developing website. Basically, A controller plays an important role in prefect working of website with efficiency. A Controller is Class where we define main business logic of website. In this Section, I will define expected controllers which will be used in my website.

1.4.1 Website Scan Controller

Functions:

- submitURL()
- startScan()
- generateReport()
- detectVulnerabilities()

1. submitURL

This controller handles the ingestion and sanitization of the target domain submitted via the frontend dashboard. It validates the URL format, checks host availability, and ensures the user has proper authorization before passing the target to the scanning pipeline.

2. ScanStart

This component manages the execution lifecycle and active telemetry of the vulnerability scanning engine. It initializes the multi-threaded crawler, dispatches audit jobs to the background task worker, and provides real-time progress updates (such as percentage completion) back to the client UI.

3. VulnerabilityDetectionController

Operating within the application core, this controller coordinates the individual fuzzing modules as they test for security flaws. It feeds crawled endpoints into the SQLi, XSS, and header

analysis engines, captures the target server's responses, and identifies matching vulnerability signatures.

4. ReportGenerationController

This controller compiles raw vulnerability data and payload captures into structured security posture documents. It aggregates findings by severity level, maps them to corresponding remediation workflows, and delivers the data required to render dashboard pie charts or downloadable reports.

-

1.4.2 Notification Controller

Functions:

- sendAlert()
- sendReportNotification()

1.5 APIs and External Libraries

API/Library	Purpose
Django	It is Backend framework of project. It provides all basic code structure. I just need to focus on building logical unit for my website.
MySQL Connector	Help in connecting with Database
OWASP ZAP API	It is main thing which provide main functionality of website. It provide vulnerability scanning engine to scan and generate insightful raw information about website.
SMTP Service	Email notifications

1.6 UML Diagrams

The following diagrams explain the working of the system:

- Use Case Diagram
- HLD Diagram

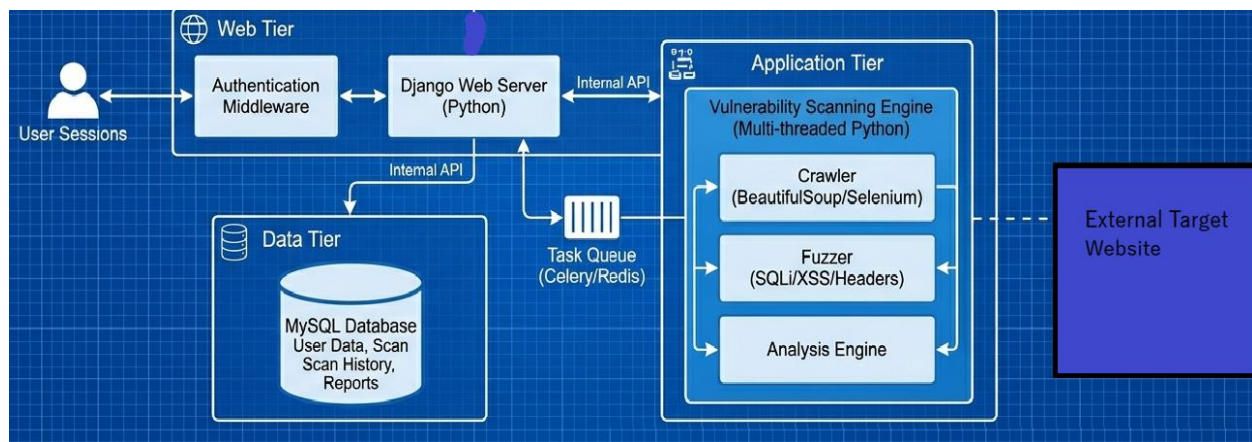
- Sequence Diagram
- ER Diagram
- Data Flow Diagram

4 diagrams are mentioned in my SRS with explanation and deployment diagram is mentioned below.

1.6.1 Level Design (HLD) Architecture Diagram

This diagram is the High-Level Design (HLD) diagram which is the structural development of the my project. It is presented as a professional schematic on a blueprint background.

- **Web Tier (Django):** The Django Web Server (Python) manages authentication, sessions, and the REST API.
- **Data Tier (MySQL):** The ACID-compliant MySQL database stores user data, scan history, and generated reports.
- **Application Tier (Scanning Engine):** This is the core multi-threaded component (implied asynchronous task management via a queue). It details the sub-modules: the Crawler (using BeautifulSoup), the Fuzzer (running specific SQLi/XSS audits), and the Analysis Engine.



Here is reference to study about those 4 diagrams.

(Reference to Diagrams).

1.7 Coding Conventions

Coding conventions are the rule of writing code for system. It describes the way/ method use in coding to declare functions, variables, constants and classes etc. If we describe each field with different conventions, then it helps in reading and debugging the code in future. These conventions directly ease the debugging but help in understanding what is written below either it is a class or constant etc.

Element	Convention
Class Names	PascalCase
Variables	snake_case
Functions	snake_case
Constants	UPPER_CASE

2. Data Model Schema

The Secure Deep Scan System uses a relational database schema to manage users, website scans, vulnerability reports, detected threats. The relational Database I am using is MySQL which is easy to implement and good for startups and small websites. More about data tables (entities) and their features are given below as:

2.1 Entities

Entities are the table in Database. These act as a blueprint for data. These entities tell what and which data to store in what which. These define rule, laws & data type for data. Following are entities which will be used in my project database.

2.1.1 Website Scan Entity

Attribute	Description
scan_id	Unique scan ID
website_url	Target website URL
scan_date	Scan timestamp

2.1.2 Scan Report Entity

Attribute	Description
report_id	Unique report ID
scan_date	Scan timestamp
risk_level	Threat severity
result_summary	Scan summary
scan_id	Linked website scan

2.1.3 Threat Detection Entity

Attribute	Description
threat_id	Unique threat ID
threat_name	Threat name
threat_type	Malware/Virus
severity	Threat severity
report_id	Linked report

2.1.4 System Log Entity

Attribute	Description
log_id	Unique log ID
activity	Activity description
log_time	Timestamp
status	Success/Failed
user_id	Linked user IP address

2.1.5 Entity Relationships

Relationship	Type
User → Uploaded File	One-to-Many
Uploaded File → Scan Report	One-to-Many
Scan Report → Threat Detection	One-to-Many
User → System Log	One-to-Many

2.2 Database Constraints

The database includes constraints to maintain data integrity.

Constraints Used in my project database are given as below:-

Constraint	Purpose
Primary Key	Unique identification
Foreign Key	Maintain relationships
NOT NULL	Prevent empty fields

Constraint	Purpose
UNIQUE	Prevent duplicate emails
CHECK	Validate data values

2.3 Data Integrity

- **ACID Compliance :**

Ensures that if a multithreaded fuzzer crashes in mid of scan, partial or corrupted report metrics are not permanently committed to the database.

- **Cascade Deletion (ON DELETE CASCADE):**

If a user deletes a recorded scan from their dashboard history, all associated crawling endpoints and vulnerability findings are cleaned up automatically via foreign key constraints, preventing orphaned rows in MySQL.

- **Indexing Optimization:**

B-Tree indexes are applied to scan_id and user_id fields to guarantee fast query response times when rendering historical data on the Django dashboard interface.

2.4 Future Database Enhancements

Future versions of the database may include:

- Admin Dashboard
- User Profile Management
- History specific to each User

3. User Interface

User Interface is the actual visual interface/page. User need visual UI to work and understand functionality provide by website. That's why today UI is necessary part of software/Web development. For my project, I will use Blue Royal Navy themed, Easy to understand UI. So , User can easily interact with website.

3.1 User Interface Design Philosophy

This project follows a modern, clean, and responsive user interface design to provide with smooth experience for users. The interface is designed according to Human Computer Interaction to ensure simplicity, accessibility, readability, and ease of navigation.

The overall UI focuses on:

- Minimal and modern design
- Easy & Understandable pages
- Only desktop responsiveness
- Fast interaction

3.2 Color Scheme

The system uses an Royal-Navy blue color palette to represent sustainability and cybersecurity/Terminal related themed.

Purpose	Color
Primary Color	Dark Royal Navy Blue
Secondary Color	Dark Light Navy Blue
Error Messages	Red

3.3 Typography

The system will use simple and readable fonts for better accessibility.

Element	Font Style
Headings	Bold Times new roman
Body Text	Regular times new roman
Buttons	Medium-weight time new roman

Font Features

- Clear readability
- Professional appearance

3.4 Responsiveness and Screen Constraints

Responsiveness is one of the major parts of a modern website. First of all, I have to explain what is responsiveness. It is a term which means how fast something responds to you back. The system is fully responsive and supports multiple screen sizes. If the backend takes time, it will still respond like loading, processing, etc. so the user does not feel that the system is hanging out or stop working.

Supported Devices

In the current version of the website, I have to decide only to support devices which are PC or Laptops. Because scanning a website is a term related to the IT field, someone who is going to scan his/her website must have a basic PC or Laptop type device.

- Laptops
- Desktop

Responsive Features

- Flexible layouts
- Adaptive navigation menus

Recommended Resolution

- 1920 × 1080 for desktop

3.5 User Interface Screens

User screen Interface is visual side of website which is visible to user and visitor of websites when they open websites. In today's world, Sometimes UI win over perfect build backend websites. Because a UI which is difficult to work with is worst experience of user which urge user to move towards the competitor. So, I have designed a basic simple and easy to interact UI so user can easily work with it and get required potential results.

3.5.1 Home Page

This is Home page. When user search website and clicks URL of website, this is first page he/she sees. This page provides all necessary information user need.

Interaction:-

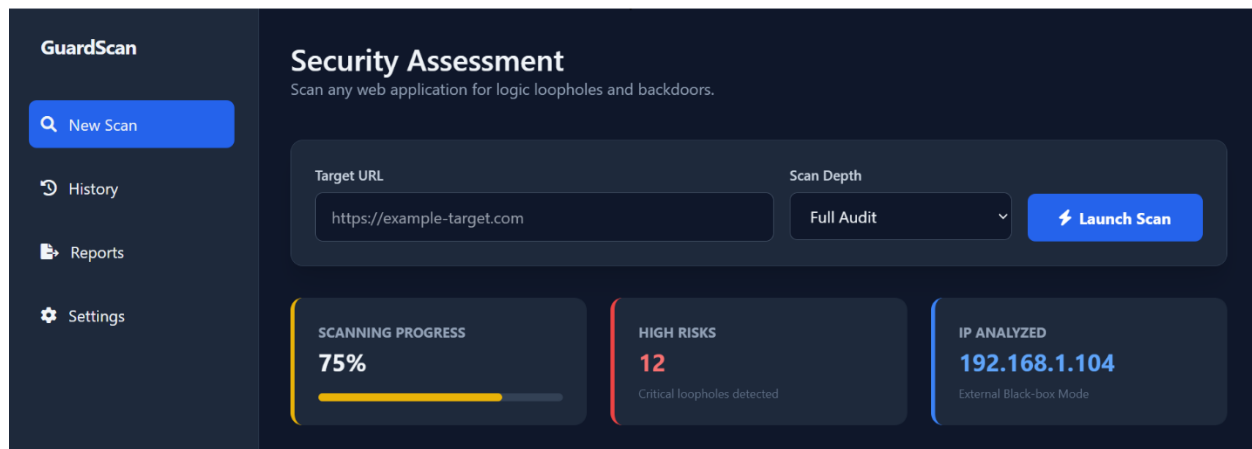
User enter URL of his/her website and press launch scan button and rest of work done by web engine to find all vulnerability of entered website.

Pre Condition :

Users must have internet.

Users must know how to copy and paste correct URL

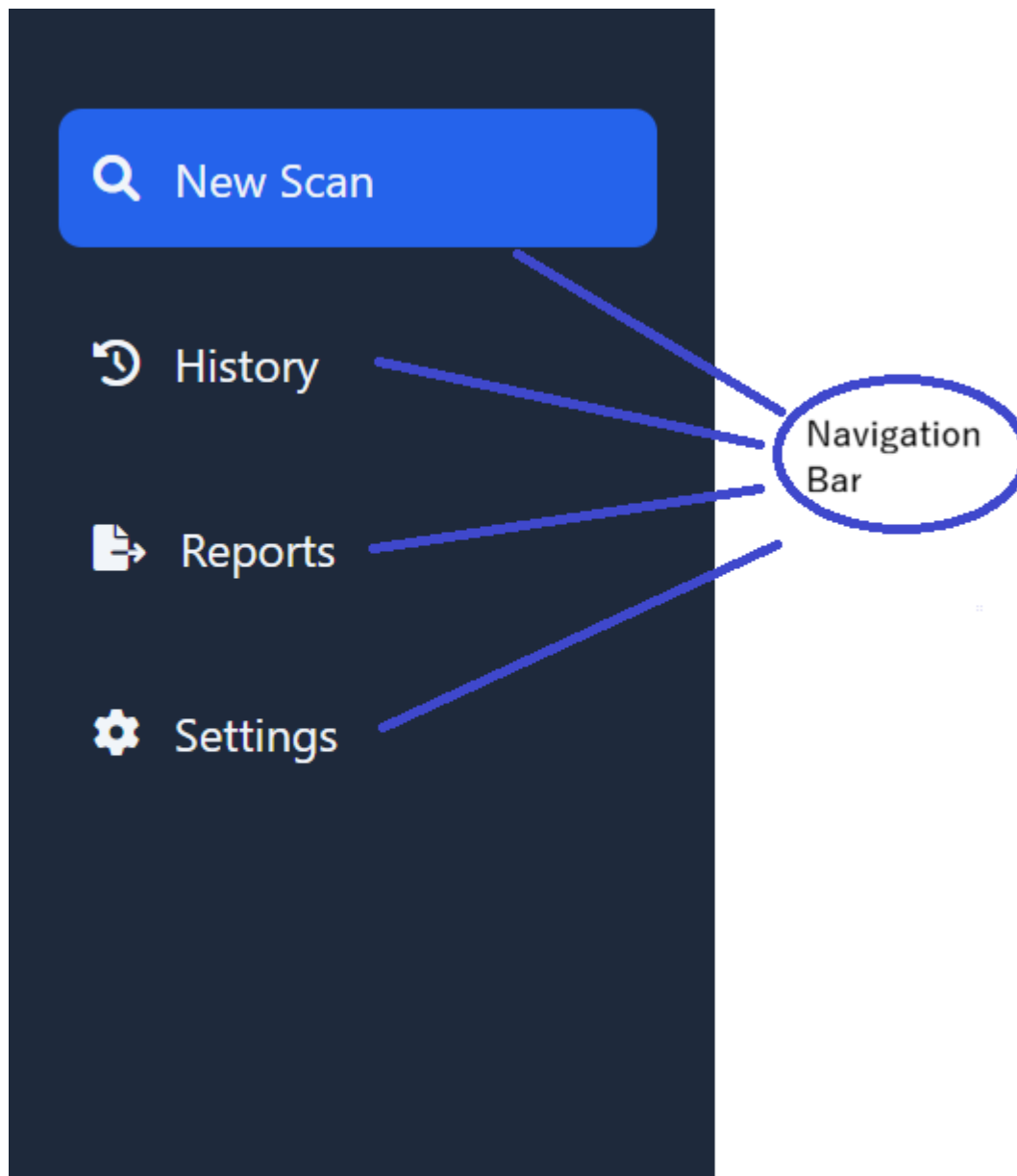
Users have basic knowledge of using Modern Website.



3.5.2 Navigation Page

The sidebar features a clean, modern dark-mode aesthetic look with four distinct options, each mapped to a core system feature:

- **New Scan:**
Highlighted as a prominent blue action button at the top, allowing users to quickly initiate an automated web vulnerability audit.
- **History:**
Dedicated to tracking past scan sessions, monitoring ongoing multi-threaded audit progress, and accessing previous target URLs.
- **Reports:**
Provides direct access to detailed security findings, including severity breakdowns and code-level remediation steps.
- **Settings:**
Allows configuration of scan parameters, user profile management, and account security options.



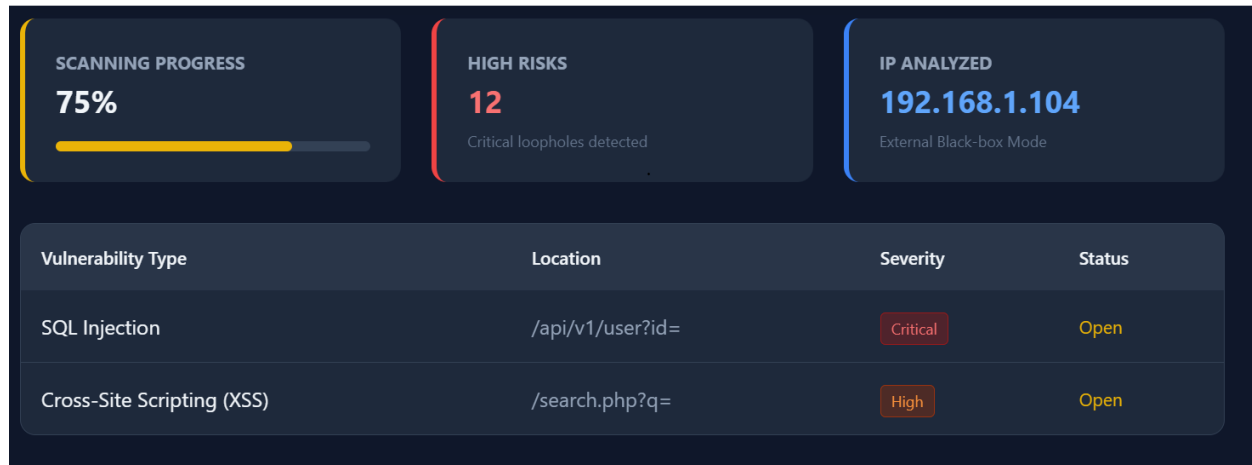
3.5.3 Report View Page

It is main part where result of target website will be uploaded. This page provide threats , vulnerabilities and their solution in a visual listing format.

Pre-Condition:

Target web scanning run successfully.

Example View of Report Data after successful scan of Website



3.9 Future UI Enhancements

Future versions may include:

- Dark mode (Black jet and silver-black Color theme)
- Voice assistance
- Mobile application interface
- AI-based personalized dashboard

These are not immediately added after release but gradually update in multiple versions.

4. References

- [OWASP Foundation, “OWASP Top 10:2021,” 2021](#)
- [OWASP Foundation, “OWASP Web Security Testing Guide v4.2,” 2020](#)

- [K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, "Technical Guide to Information Security Testing and Assessment," NIST Special Publication 800-115, Gaithersburg, MD, USA, 2008.\[3\] MITRE Corporation, "Common Weakness Enumeration," 2026](#)
- [D. Stuttard and M. Pinto, The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws, 2nd ed. Indianapolis, IN, USA: Wiley, 2011.\[5\] FIRST.Org, Inc., "Common Vulnerability Scoring System version 3.1: Specification Document," 2019](#)
- [L. Richardson, "Beautiful Soup Documentation," 2024](#)